Once the integration works, we create our chatbot with a custom prompt where LangChain will pass the context and the question, as shown in *Figure 6.21*.

> **Modifying your Prompt**
>
> Please pay attention to the prompt because you will have to modify it according to your use cases. For this project, we create a simple RetrievalQA chain without a conversation history buffer or complex RAG chains. The idea is to familiarize you with tools that can be applied to various scenarios on your own using domain expertise in RAG.

```
import langchain
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langchain.chat_models import ChatDatabricks

# Using Foundational Model from Databricks
chat_model = ChatDatabricks(endpoint="databricks-mixtral-8x7b-instruct",
                            max_tokens = 200
                            )

TEMPLATE = """
You are an assistant for the AI Swat Team. You are answering questions related to
the GenerativeAI and LLM's and how they impact humans life, labour, economic and
financial impact. If the question is not related to one of these topics, kindly
decline to answer. If you don't know the answer, just say that you don't know, don't
try to make up an answer. Keep the answer as concise as possible. Use the following
pieces of context to answer the question at the end:
{context}
Question: {question}
Answer:
"""
prompt = PromptTemplate(template=TEMPLATE, input_variables=["context", "question"])

chain = RetrievalQA.from_chain_type(
    llm=chat_model,
    chain_type="stuff",
    retriever=get_retriever(),
    chain_type_kwargs={"prompt": prompt}
)
```

Figure 6.21 – Creating a RetrievalQA chain with a custom prompt